

First pass - getting all labels and sorting instructions

The `SortInstructionsAndGetLabels` class prepares the instructions for execution and identifies labels in the first pass of the program. Upon instantiation, it gets all 'instruction' elements from the `DOMDocument` and stores them in the `instructions` array, indexed by their 'order' attribute. If the order is equal or less than 0, the program writes to `stderr` and exits with error code. The instructions are then sorted in ascending order by their 'order' attribute.

The `execute` method of the class is where the first pass of the program happens. It iterates over each instruction in the `instructions` array. If an instruction has an 'opcode' attribute of 'LABEL', a new `Label` object is created and executed. The `Label` object stores the value in `MemoryManager`. The method then returns an associative array containing the total number of instructions and the sorted instructions.

Second pass - execution of the instructions

In the second pass, the program loops from 1 to the highest order of instruction. It checks if an instruction exists for the current iteration number. If not, it skips to the next iteration. The program gets all arguments of the instruction and adds them to the `allArgs` array, then it sorts the arguments by their `nodeName`.

After that, it enters a switch case in which, depending on the instruction's opcode attribute, it creates a new class. If the opcode is not valid, it writes to `stderr` and exits with an error code.

MemoryManager class

The `MemoryManager` class, designed as a Singleton, manages the program's memory. It manages frames (Global, Local, Temporary), the data stack, call stack, and labels with their orders.

The `getInstance` method ensures a single `MemoryManager` instance throughout the program. Frame operations are handled by methods like `createFrame`, `pushFrame`, and `popFrame`, while stack operations are managed by `pushs`, `pops`, `pushCall`, and `popCall`.

Variable access and manipulation within frames are done by `getVariableInFrame`, `setVariableInFrame`, and `doesVariableExistInFrame`. Label management is achieved through `doesLabelExist`, `setLabel`, and `getLabelOrder`.

The `getFramesStatus` method provides a status of all frames, including variables, types, and values. Instruction order is managed by `setOrderOfInstruction` and `getOrderOfInstruction`.

Abstract classes

The program uses several abstract classes. `Opcode` class serves as a foundation for all other opcodes that extend this class. It has common properties and methods that all opcodes will use. The `execute` method is declared but not implemented, each subclass will implement their own `execute` method.

The `Relational`, `Boolean`, and `Arithmetic` classes are abstract classes that extend the `Opcode` class. Each provides a structure for a specific type of operation. The `Relational` class is designed for relational operations such as equality and inequality checks. The `Boolean` class is for boolean operations like AND and OR, while the `Arithmetic` class is designed for arithmetic operations including addition, subtraction, multiplication, and division.

Each of these three classes declare operation method, that subclasses implement and the `result` is determined by the class operation.

Utility classes

The `GetValueAndType` class is a utility class that extracts the value and type from a `DOMElement` node. Its `execute` method takes a `DOMElement` node and a `MemoryManager` instance as parameters.

If the node represents a variable (type attribute is 'var'), the method splits the node value into `frame` and `variable` and retrieves the stored data for this variable from the specified frame. If the stored data's type is also 'var', the method recursively retrieves the value and type from the nested variable.

If the node represents a literal value (type attribute is not 'var'), the method directly retrieves the value from the node and the type from the type attribute.

Control Flow operation codes

The `Jump`, `JumpIfEq`, and `JumpIfNeq` classes are key in controlling the flow of the program. They get the label to jump to from the operand's `nodeValue`. The `JumpIfEq` and `JumpIfNeq` classes first evaluate their respective conditions. If the condition is met, they store the order number of the instruction to jump to in the `order` property. This order number is then assigned to the control variable `i` of the loop, effectively executing the jump.

The `Call` class gets the label to jump to from the operand's `nodeValue`. It then stores the current instruction order in the memory manager's call stack before updating the `order` property with the order of the label.

The `ReturnOp` class, on the other hand, pops the last order from the memory manager's call stack and assigns it to the `order` property. This effectively returns the flow of the program to the instruction following the last call.

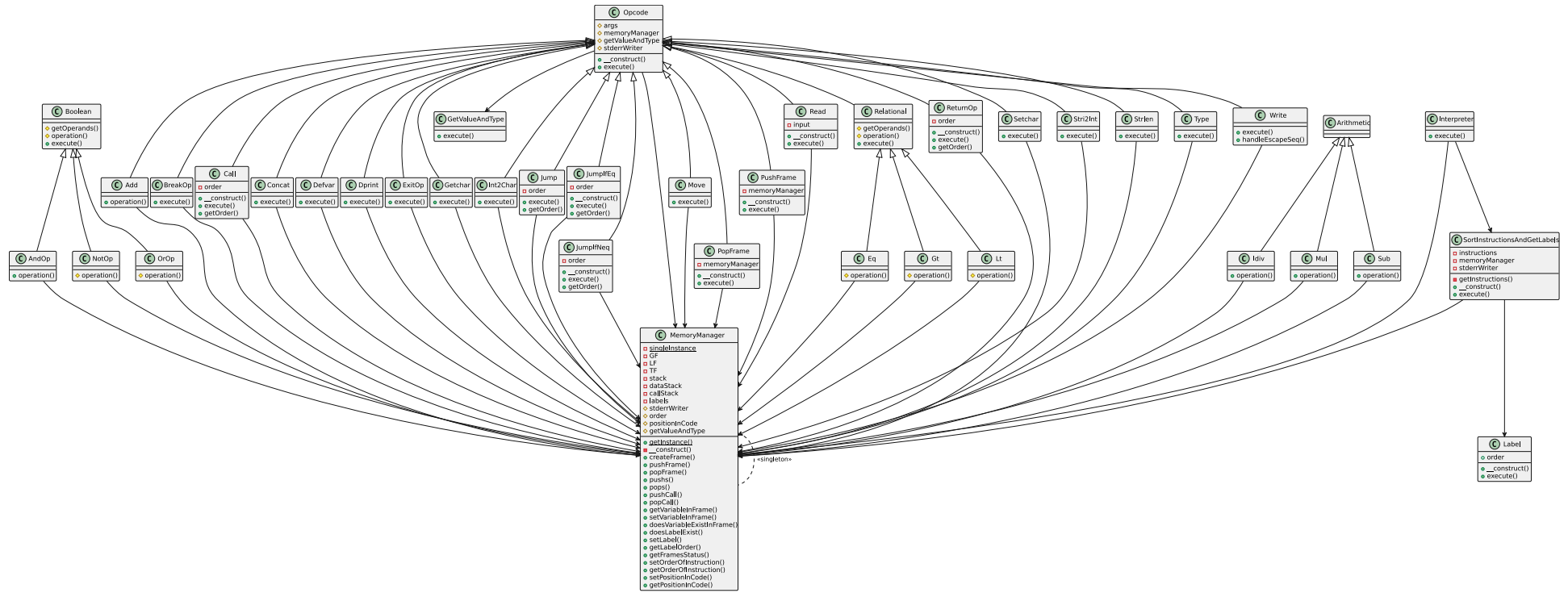
Read and Write classes

The `Read` class is designed to read input and store it in a variable. It validates the input type, reads the input, checks for null values, verifies the type, and stores the value in a specified frame.

The `Write` class, on the other hand, is used to write a value to the standard output. It retrieves the value to write, checks the instruction type, recasts the value if necessary, and writes the value to the output. It also includes a method to handle escape sequences in strings.

Error Handling

Error handling in the program is accomplished through various types of checks. When unexpected behavior is encountered, an error message is written to `stderr` using the built-in core class, `StreamWriter`. After that, the program exits with an appropriate error code.



UML Diagram